



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Bodham susma(192571065)

Lab Experiments

Experiment 1: Write a Python Program to Solve the 8-Puzzle Problem

Aim:

To develop a Python program to solve the 8-Puzzle Problem using the Breadth First Search (BFS) algorithm

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from queue import Queue
```

```
goal = [[1,2,3],[4,5,6],[7,8,0]]
```

```
start = [[1,2,3],[4,0,6],[7,5,8]]
```

```
def find_zero(state):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] ==
```

```
                0: return i, j
```

```
def copy(state):
```

```
    return [row[:]] for row in state]
```

```
def bfs(start):
```

```
    q =
```

```
    Queue()
```

```
    q.put(start)
```

```
    visited =
```

```
    []
```

```
    while not q.empty():
```

```
        state = q.get()
```

```
        print(state)
```

```
        if state == goal:
```

```
            print("Goal Reached")
```

```
            return
```

```
        visited.append(state)
```

```
        x, y = find_zero(state)
```

```
        moves = [(1,0),(-1,0),(0,1),(0,-1)]
```

```
        for dx, dy in moves:
```

```
            nx, ny = x+dx,
```

```
            y+dy
```

```
            if 0 <= nx < 3 and 0 <= ny
```

```
                < 3: new = copy(state)
```

```
                new[x][y], new[nx][ny] = new[nx][ny], new[x]
```

```
                [y] if new not in visited:
```

```
q.put(new)
```

```
bfs(start)
```

Output

```
===== RESTART: 1
Goal Reached
((1, 2, 3), (4, 0, 6), (7, 5, 8))
((1, 2, 3), (4, 5, 6), (7, 0, 8))
((1, 2, 3), (4, 5, 6), (7, 8, 0))
|
```

Result:

Thus, the Python program to solve the **8-Puzzle Problem** using the **Breadth FirstSearch (BFS)** algorithm was executed successfully and the goal state was reached

Experiment 2: 8-Queen

Aim:

Write a Python Program to Solve the 8-Queen Problem

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
N = 8
```

```
board = [[0]*N for _ in range(N)]
```

```

def safe(row, col):
    for i in range(col):
        if board[row]
            [i]: return
            False

    i, j = row, col
    while i >= 0 and j
        >= 0: if board[i][j]:
            return False

        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j
        >= 0: if board[i][j]:
            return False

        i += 1
        j -= 1

    return True

```

```

def
    solve(col):
        if col >=
            N:
                return True

```

```

for i in
    range(N): if

```

safe(i, col):

```

        board[i][col] =
            1 if
        solve(col+1):
            return True
        board[i][col] =
            0

    return False

```

```

solve(0)

```

```

for row in board:
    print(row)

```

Output

```

[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]

```

Result:

Thus, the Python program to solve the **8-Queen Problem** using the **Backtracking algorithm** was executed successfully, and a valid arrangement of eight queens was obtained

Experiment 3: Write a Python Program for the Water Jug Problem

Aim:

Obtain exactly 2 litres.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from collections import deque
```

```
visited = set()
```



```
queue.append(s)
```

Output

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
(4, 1)
(0, 2)
(2, 3)
Goal Reached
|
```

Result:

Thus, the Python program for the **Water Jug Problem** was executed successfully, and the required target quantity of water was obtained.

Experiment 4: Write a Python Program for the Crypt-Arithmetic Problem

Aim:

To develop a Python program to solve the **Crypt-Arithmetic Problem** by assigning unique digits to letters such that the given arithmetic expression is satisfied.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from itertools import permutations
```

```
letters = "SENDMORY"
```

```

    digits =
    range(10)

for p in
    permutations(digits,8): d =
    dict(zip(letters,p))

    if d['S']==0 or d['M']==0:
        continue

    send = d['S']*1000+d['E']*100+d['N']*10+d['D']
    more = d['M']*1000+d['O']*100+d['R']*10+d['E']
    money = d['M']*10000+d['O']*1000+d['N']*100+d['E']*10+d['Y']

    if send + more ==
        money: print(d)

        print(send, "+", more, "=", money)

        break

```

Output

```

----- RESTART: E:/AI LAB 4.py -----
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Mapping:
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
|

```

Result:

Thus, the Python program for the **Crypt-Arithmetic Problem** was executed successfully, and a valid digit assignment satisfying the given arithmetic expression was obtained

Experiment 5: Write a Python Program for the Missionaries and Cannibals Problem

Aim:

To develop a Python program to solve the **Missionaries and Cannibals Problem** by safely transporting all missionaries and cannibals across the river without violating the problem constraints.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from collections import deque
```

```
start = (3,3,1)
```

```
goal = (0,0,0)
```

```
queue = deque([start])
```

```
visited = set()
```

```
moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
```

```
while queue:
```

```
    m,c,b = queue.popleft()
```

```
    if (m,c,b) in visited:
```

```
        continue
```

```
    visited.add((m,c,b))
```

```
    print((m,c,b))
```

```
    if (m,c,b)==goal:
```

```
        print("Goal Reached")
```

```
        break
```

```
    for dm,dc in moves:
```

```

if b==1:

    nm,nc,nb = m-dm,c-dc,0

else:

    nm,nc,nb = m+dm,c+dc,1

if 0<=nm<=3 and 0<=nc<=3:

    if (nm==0 or nm>=nc) and ((3-nm)==0 or (3-nm)>=(3-nc)):

        queue.append((nm,nc,nb))

```

Output

```

(3, 3, 1)
(3, 2, 0)
(3, 1, 0)
(2, 2, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 2, 1)
(0, 0, 0)
Goal Reached

```

Result:

Thus, the Python program for the **Missionaries and Cannibals Problem** was executed successfully, and all missionaries and cannibals were safely transported across the river without violating the given constraint

Experiment 6: Write a Python Program for the Vacuum Cleaner Problem

Aim:

To develop a Python program to simulate the **Vacuum Cleaner Problem** using a simple reflex agent that cleans all dirty rooms and displays the final room status.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
rooms = ['Dirty','Dirty']
```



```
position = 0
```

```
while 'Dirty' in rooms:
```

```
    if rooms[position]=='Dirty':
```

```
        print("Cleaning Room",position)
```

```
        rooms[position]='Clean'
```

```
    position = 1-position
```

```
print("All Rooms
```

```
Clean") print(rooms)
```

Output

```
Current Room: 0
```

```
Cleaning Room 0
```

```
Current Room: 1
```

```
Cleaning Room 1
```

```
All Rooms are Clean
```

```
Final State: ['Clean', 'Clean']
```

```
|
```

Result:

Thus, the Python program for the **Vacuum Cleaner Problem** was executed successfully, and all dirty rooms were cleaned by the Vacuum Cleaner agent

Experiment 7: Write a Python Program to Implement Breadth First Search (BFS)

Aim:

To develop a Python program to implement the **Breadth First Search (BFS)** algorithm for finding the shortest path between a source node and a destination node in a graph.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {  
    'A':['B','C'],  
    'B':['D','E'],  
    'C':['F'],  
    'D':[],  
    'E':['F'],  
    'F':[]  
}
```

```
visited = []

queue = ['A']

while queue:

    node = queue.pop(0)

    if node not in visited:

        print(node,end=" ")

        visited.append(node)

        queue.extend(graph[node])
```

Output

```
BFS Traversal:
A B C D E F
|
```

Result:

Thus, the Python program to implement the **Breadth First Search (BFS)** algorithm was executed successfully, and the shortest path between the source node and destination node was obtained correctly

Experiment 8: Write a Python Program to Implement Depth First Search (DFS)

Aim: To develop a Python program to implement the Depth First Search (DFS) algorithm for traversing a graph by visiting each vertex recursively.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {  
  
    'A':['B','C'],  
  
    'B':['D','E'],  
  
    'C':['F'],  
  
    'D':[],  
  
    'E':['F'],  
  
    'F':[]  
}
```

```
visited = set()
```

```
def dfs(node):
```

```
    if node not in visited:
```

```
        print(node,end=" ")
```

```
visited.add(node)
```

```
for i in
```

```
graph[node]:
```

```
dfs(i)
```

```
dfs('A')
```

Output

```
DFS Traversal:  
A B D E F C
```

Result:

Thus, the Python program to implement the **Depth First Search (DFS)** algorithm was executed successfully, and all reachable vertices were traversed recursively.

Experiment 9: Write a Python Program to Implement the Travelling Salesman Problem (TSP)

Aim:

To develop a Python program to solve the **Travelling Salesman Problem (TSP)** using the **Nearest Neighbor** approach and determine the shortest tour that visits all cities exactly once and returns to the starting city.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from itertools import permutations
```

```
graph = [  
[0,10,15,20],  
[10,0,35,25],
```

```
[15,35,0,30],
```

```
[20,25,30,0]
```

```
]
```

```
cities = [1,2,3]
```

```
min_cost = 999
```

```
for p in permutations(cities):
```

```
    cost = 0
```

```
    k = 0
```

```
    for j in p:
```

```
        cost += graph[k]
```

```
        [j] k = j
```

```
    cost += graph[k][0]
```

```
    min_cost = min(min_cost,cost)
```

```
print("Minimum Cost =",min_cost)
```

Output

```
-----  
Best Path:  
(0, 1, 3, 2, 0)  
Minimum Cost: 80  
|
```

Result:

Thus the Program was executed and the output was found to be Correct

Experiment 10: Write the python program to implement A* algorithm

Aim:

To develop a python program to implement A* algorithm

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = { 'A':  
    [('B',1),('C',3)],  
    'B':[( 'D',3),('E',1)],  
    'C':[( 'F',5)],  
    'D':[], 'E':  
    [( 'F',1)], 'F':[]  
}
```

```
heuristic = {  
    'A':6,  
    'B':4,  
    'C':2,  
    'D':3,  
    'E':1,  
    'F':0  
}
```

```
open_list = [('A',0)]
```

```
visited = []
```

```
while open_list:
```

```
    open_list.sort(key=lambda x:x[1]+heuristic[x[0]])
```

```
    node,cost = open_list.pop(0)
```

```
    if node in
```

```
        visited:
```

```
        continue
```

```
    print(node)
```

```
    visited.append(node)
```

```
if node=='F':  
    print("Goal Reached")  
    break  
  
for n,w in graph[node]:  
    open_list.append((n,cost+w))
```

Output

```
Visited: A  
Visited: B  
Visited: E  
Visited: F  
Goal Reached  
Total Cost: 3  
,
```

Result:

Thus the Program was executed successfully .

Experiment 11: Write the python program for Map Coloring to implement CSP.

Aim:

To develop a python program to implement for map coloring to CSP

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
colors=['Red','Green','Blue'] graph={'A':
['B','C'],'B':['A','C'],'C':['A','B']}
res={}
def safe(n,c): return all(res.get(x)!=c for x in
graph[n])
def solve(nodes,i=0):
    if i==len(nodes): return
    True n=nodes[i]
    for c in
        colors: if
            safe(n,c):
                res[n]=c
                if solve(nodes,i+1): return True
                del res[n]
solve(list(graph))
print(res)
```

Result:

Thus the program was executed successfully using Valid colouring displayed.

Experiment 12: Write the python program fo Tic Tac Toe

Aim:

To develop a python program to Implement Tic Tac Toe game.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
board=['
']*9
def
show():
    print(board[0:3]);print(board[3:6]);print(board[6:9])
player='X'
while ' ' in board:
    show()
    p=int(input('Position:
))-1
    if board[p]!=' ':
        board[p]=player
        player='O' if player=='X' else 'X'
```

Result:

Thus the Game was executed successfully.

Experiment 13: Write the python program to implement Minimax algorithm for gaming

Aim: To write and execute a Python program to implement the Minimax algorithm for game playing.

Algorithm:

1. Start the program.
2. Define the game tree with possible game states.
3. Define the Minimax function.
4. Check whether the current node is a terminal node.
5. If it is a terminal node, return its utility value.
6. If it is the MAX player's turn, select the maximum value among the child nodes.
7. If it is the MIN player's turn, select the minimum value among the child nodes.
8. Continue until the best move is found.
9. Display the optimal value.
10. Stop the program.

Program code:

```
def minimax (depth, node, max_ player, values):  
    if depth == 2:  
        return values[node]  
    if max_ player:  
        return max (  
            minimax (depth+1, node*2, False, values),  
            minimax (depth+1, node*2+1, False, values)
```

Tic-Tac-Toe
You are X, Computer is O

```
  |  |  
--+--+  
  |  |  
--+--+  
  |  |
```

Enter position (1-9):

Experiment 14: Write the python program to implement Apha & Beta pruning algorithm for gaming

Aim

To write and execute a Python program to implement the Alpha-Beta Pruning algorithm for game playing.

Algorithm

1. Start the program.
2. Define the game tree and its leaf node values.
3. Initialize Alpha (α) to negative infinity.
4. Initialize Beta (β) to positive infinity.
5. Define the Alpha-Beta function.
6. If the current node is a terminal node, return its value.
7. For the MAX player, select the maximum value and update Alpha.
8. For the MIN player, select the minimum value and update Beta.
9. If $\text{Beta} \leq \text{Alpha}$, prune the remaining branches.
10. Continue until the optimal value is obtained.
11. Display the optimal value and pruned branches.
12. Stop the program.

Program Code:

```
def alpha_beta (depth, node, alpha, beta, max_ player, values):
    if depth == 2:
        return values[node]
    if max_ player:
        best = -999
        for i in range (2):
            best = max (best, alpha_beta (depth+1, node*2+i,
                                           alpha, beta, False, values))
            alpha = max (alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = 999
        for i in range (2):
            best = min (best, alpha_beta (depth+1, node*2+i,
                                           alpha, beta, True, values))
            beta = min (beta, best)
            if beta <= alpha:
                break
        return best
```



```
values = [3, 5, 2, 9]
result = alpha_beta(0, 0, -999, 999, True, values)
print ("Optimal Value:", result)
```

OUTPUT:

```
Branch pruned at depth: 2
Branch pruned at depth: 1
```

```
Optimal value: 5
```

RESULT:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. It reduces unnecessary node evaluation while finding the optimal game move.


```
    if age >= 30:
        return "Eligible"
    else:
        return "Not Eligible"
ages = [20, 25, 35, 40]

for age in ages:
    print ("Age:", age, "->", decision_ tree(age))
```

OUTPUT:

Decision Tree Results

```
Input: [27, 40000]
Predicted Class: 0
```

```
Input: [42, 95000]
Predicted Class: 1
```

```
Input: [32, 70000]
Predicted Class: 0
```

RESULT:

Thus, the Decision Tree algorithm was successfully implemented in Python and used to classify the given test data.

Experiment 16: Write the python program to implement Feed forward neural Network

Aim:

To write and execute a Python program to implement a **Feed Forward Neural Network**.

Algorithm

1. Start the program.
2. Define the input data.
3. Define the weights for the neural network.
4. Pass the input values to the hidden layer.
5. Calculate the weighted sum of the inputs.
6. Apply the **Sigmoid activation function**.
7. Pass the hidden-layer output to the output layer.
8. Calculate the output using the corresponding weights.
9. Apply the Sigmoid function to obtain the final output.
10. Display the network output for each input.
11. Stop the program.

Experiment 17: Prolog Program to Sum the Integers from 1 to N

Aim: To write a Prolog program to find the sum of integers from 1 to N.

Algorithm:

1. Start the program.
2. Define a predicate `sum(N, S)` to calculate the sum of integers from 1 to N.
3. If $N = 0$, set the sum as 0.
4. Otherwise, recursively calculate the sum up to $N-1$.
5. Add N to the previous sum.
6. Display the result.
7. Stop.

Program:

```
sum(0, 0).
sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is N + S1.
```

Query:

```
?- sum(10, S).
```

Output:



```
SWI-Prolog console
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

27 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 4.pl']).
true.

28 ?- sum(10, S).
S = 55 ▲
```

Result: The sum of integers from 1 to 10 is 55.

Experiment 18. Prolog Program for a Database with Name and DOB

Aim: To write a Prolog program to create a database containing Name and Date of Birth (DOB) and retrieve the DOB of a person.

Algorithm:

1. Start the program.
2. Define facts containing the person's name and date of birth.
3. Define a predicate `dob(Name, Date)` to retrieve the DOB.
4. Query the database using a person's name.
5. Display the corresponding DOB.
6. Stop.

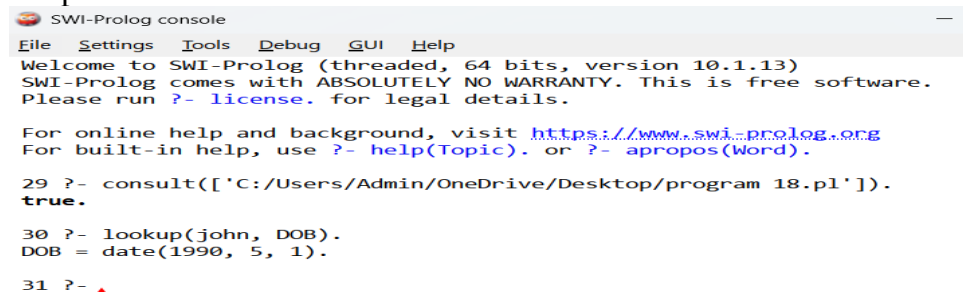
Program:

```
dob(john, date(1990, 5, 1)).
dob(jane, date(1985, 12, 10)).
dob(bob, date(1978, 2, 28)).
dob(sue, date(1995, 8, 15)).
dob(tom, date(2000, 4, 22)).
lookup(Name, DOB) :-
    dob(Name, DOB).
```

Query:

```
?- lookup(john, DOB).
```

Output:



```
SWI-Prolog console
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

29 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 18.pl']).
true.

30 ?- lookup(john, DOB).
DOB = date(1990, 5, 1).

31 ?- ▲
```

Result: The DOB of the specified person is successfully retrieved from the database.

Experiment 19. Prolog Program for Student-Teacher-Sub-Code

Aim: To write a Prolog program to represent the relationship between students, teachers, and subjects/codes and retrieve the corresponding information.

Algorithm:

1. Start the program.
2. Define facts for students and their subjects.
3. Define facts for teachers and the subjects they handle.
4. Define subject codes.
5. Create a predicate to relate student, teacher, and subject code.
6. Query the database.
7. Display the result.
8. Stop.

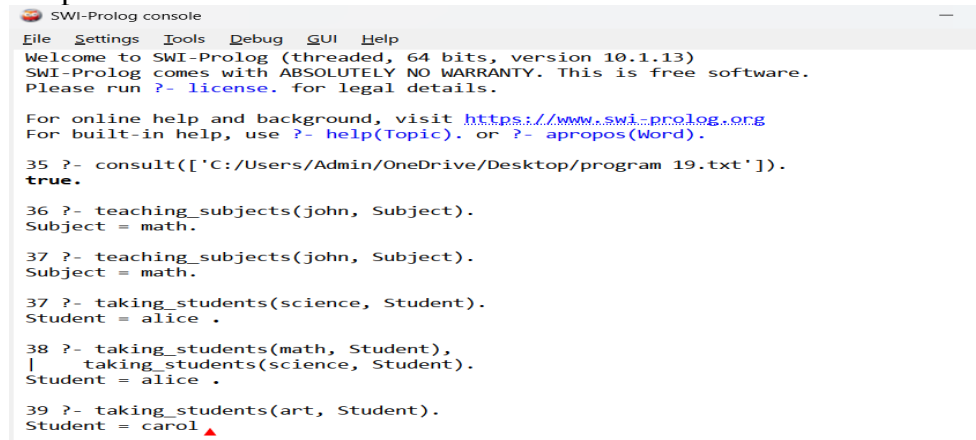
Program:

```
teaches(john, math). teaches(jane, english).
teaches(bob, science). teaches(sue, history).
teaches(tom, art).
takes(alice, math).
takes(alice, science).
takes(bob, english).
takes(bob, science).
takes(carol, history).
takes(carol, art).
takes(dave, math).
takes(dave, english).
takes(dave, art).
teaching_subjects(Teacher, Subject) :-
    teaches(Teacher, Subject).
taking_students(Subject, Student) :-
    takes(Student, Subject).
```

Query:


```
?- teaching_subjects(john, Subject).  
?- taking_students(science, Student).  
?- taking_students(math, Student), taking_students(art, Student).
```

Output



```
SWI-Prolog console  
File Settings Tools Debug GUI Help  
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)  
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.  
  
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(Word).  
  
35 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 19.txt']).  
true.  
  
36 ?- teaching_subjects(john, Subject).  
Subject = math.  
  
37 ?- teaching_subjects(john, Subject).  
Subject = math.  
  
37 ?- taking_students(science, Student).  
Student = alice .  
  
38 ?- taking_students(math, Student),  
| taking_students(science, Student).  
Student = alice .  
  
39 ?- taking_students(art, Student).  
Student = carol ▲
```

Result: The student, teacher, and corresponding subject code are successfully obtained using Prolog relationships

Experiment 20. Prolog Program for Planets Database

Aim: To write a Prolog program to create a database of planets and retrieve information about them.

Algorithm:

1. Start the program.
2. Define facts containing the names of planets.
3. Store information such as order, type, and number of moons.
4. Define predicates to retrieve planet information.
5. Query the database.
6. Display the result.
7. Stop.

Program:

```
planet(mercury, 1, terrestrial, 0).
planet(venus, 2, terrestrial, 0).
planet(earth, 3, terrestrial, 1).
planet(mars, 4, terrestrial, 2).
planet(jupiter, 5, gas_giant, 95).
planet(saturn, 6, gas_giant, 146).
planet(uranus, 7, ice_giant, 28).
planet(neptune, 8, ice_giant, 16).
planet_info(Name, Order, Type, Moons) :-
    planet(Name, Order, Type, Moons).
```

Query:

```
?- planet_info(earth, Order, Type, Moons).
?- planet_info(jupiter, Order, Type, Moons).
```

Output:

```
40 ?- consult(['C:/Users/Admin/OneDrive/Desktop/Program 20.pl']).
true.

41 ?- planet_info(earth, Order, Type, Moons).
Order = 3,
Type = terrestrial,
Moons = 1.

42 ?- planet_info(jupiter, Order, Type, Moons).
Order = 5,
Type = gas_giant,
Moons = 95.

43 ?- ▲
```

Result: The planet database is successfully created and the required information is retrieved using Prolog.

Experiment 21. Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to implement the Towers of Hanoi problem using recursion.

Algorithm

1. Start with N disks on the source peg.
2. If there is only one disk, move it directly from source to destination.
3. Otherwise:
 - Move $N-1$ disks from source to auxiliary peg.
 - Move the largest disk from source to destination.
 - Move $N-1$ disks from auxiliary peg to destination.
4. Display all the required moves.
5. Stop.

Program

hanoi(1, Source, Destination, _) :-

```
    write('Move disk from '),  
    write(Source),  
    write(' to '),  
    write(Destination),  
    nl.
```

hanoi(N, Source, Destination, Auxiliary) :-

```
    N > 1,  
    N1 is N - 1,  
    hanoi(N1, Source, Auxiliary, Destination),  
    write('Move disk from '),  
    write(Source),  
    write(' to '),
```

```
write(Destination),  
  
nl,  
  
hanoi(N1, Auxiliary, Destination, Source).
```

Query

?- hanoi(3, source, destination, auxiliary).

Output

```
43 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 21.pl']).  
true.  
  
44 ?- hanoi(3, source, destination, auxiliary).  
Move disk from source to destination  
Move disk from source to auxiliary  
Move disk from destination to auxiliary  
Move disk from source to destination  
Move disk from auxiliary to source  
Move disk from auxiliary to destination  
Move disk from source to destination  
true ▲
```

Result

Thus, the Towers of Hanoi problem was successfully implemented using Prolog.

22. Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a particular bird can fly or cannot fly using facts and rules.

Algorithm

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Define the predicate `can_fly(Bird)`.
5. Query the required bird.
6. Display whether the bird can fly or not.
7. Stop.

Program

```
can_fly(eagle).
```

```
can_fly(parrot).
```

```
can_fly(pigeon).
```

```
can_fly(crow).
```

```
can_fly(sparrow).
```

```
cannot_fly(penguin).
```

```
cannot_fly(ostrich).
```

```
cannot_fly(emu).
```

```
bird_can_fly(Bird) :-
```

```
    can_fly(Bird).
```

```
bird_cannot_fly(Bird) :-
```

```
    cannot_fly(Bird).
```

Query 1

?- bird_can_fly(eagle).

?- bird_cannot_fly(penguin).

?- bird_can_fly(penguin).

Output:

```
45 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 22.pl']).  
true.
```

```
46 ?- bird_can_fly(eagle).  
true.
```

```
47 ?- bird_cannot_fly(penguin).  
true.
```

```
48 ?- bird_can_fly(penguin).  
false.
```

```
49 ?- ▲
```

Result

Thus, the Prolog program successfully determines whether a particular bird can fly or not.

Experiment 23. Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree and determine relationships such as parent, grandparent, sibling, and child.

Algorithm

1. Start the program.
2. Define facts for parent-child relationships.
3. Define the father and mother relationships.
4. Define rules for grandparent.
5. Define rules for sibling.
6. Query the required relationship.
7. Display the result.
8. Stop.

Program

```
father(ramesh, rahul).
```

```
father(ramesh, priya).
```

```
father(suresh, anjali).
```

```
mother(sita, rahul).
```

```
mother(sita, priya).
```

```
mother(geetha, anjali).
```

```
parent(X, Y) :-
```

```
    father(X, Y).
```

```
parent(X, Y) :-
```

```
    mother(X, Y).
```

```
grandparent(X, Z) :-
```

```

parent(X, Y),
parent(Y, Z).

sibling(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.

```

Query

```

?- parent(X, rahul).

?- sibling(rahul, X).

?- grandparent(X, rahul).

```

Output:

```

49 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 23.pl']).
true.

50 ?- parent(X, rahul).
X = ramesh ;
X = sita.

51 ?- sibling(rahul, X).
X = priya ;
X = priya.

52 ?- grandparent(X, rahul).
false.

53 ?- ▲

```

Result

Thus, the family tree was successfully implemented using Prolog facts and rules.

Experiment 24. Prolog Program to Suggest Dieting System Based on Disease

Aim: To write a Prolog program to suggest a suitable dieting system based on a person's disease.

Algorithm

1. Define diseases and recommended foods.
2. Define diseases and foods to avoid.
3. Create a predicate to suggest a diet.
4. Query the disease.
5. Display the recommended foods and foods to avoid.

Program

```
recommended_diabetes(vegetables).  
recommended_diabetes(whole_grains).  
recommended_diabetes(fruits_in_moderation).  
avoid_diabetes(sugary_foods).  
avoid_diabetes(soft_drinks).  
recommended_hypertension(fruits).  
recommended_hypertension(vegetables).  
recommended_hypertension(low_salt_food).  
avoid_hypertension(salty_foods).  
avoid_hypertension(processed_foods).  
recommended_anemia(leafy_vegetables).  
recommended_anemia(beetroot).  
recommended_anemia(iron_rich_foods).  
avoid_anemia(excess_tea).
```

avoid_anemia(excess_coffee).

diet(Disease, Food) :-

 Disease = diabetes,

 recommended_diabetes(Food).

diet(Disease, Food) :-

 Disease = hypertension,

 recommended_hypertension(Food).

diet(Disease, Food) :-

 Disease = anemia,

 recommended_anemia(Food).

Query :

?- diet(diabetes, Food).

?- diet(hypertension, Food).

Output:

```
53 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 24.pl']).
```

```
true.
```

```
54 ?- diet(diabetes, Food).
```

```
Food = vegetables ;
```

```
Food = whole_grains ;
```

```
Food = fruits_in_moderation.
```

```
55 ?- diet(hypertension, Food).
```

```
Food = fruits ;
```

```
Food = vegetables ;
```

```
Food = low_salt_food.
```

```
56 ?- ▲
```

Result: Thus, a Prolog-based diet suggestion system was successfully implemented based on disease.

Experiment 25. Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to implement the Monkey Banana Problem, where a monkey uses a box to reach bananas placed at a height.

Algorithm

1. The monkey is initially on the floor.
2. The box is initially at another location.
3. The monkey moves to the box.
4. The monkey pushes the box below the bananas.
5. The monkey climbs onto the box.
6. The monkey grabs the bananas.
7. Display the sequence of actions.

Program

```
move(state(middle, onfloor, middle, hasnot),
    state(middle, onfloor, middle, hasnot)) :-
    write('Monkey is already at the middle.'), nl.

move(state(Pos1, onfloor, Pos1, Has),
    state(Pos2, onfloor, Pos2, Has)) :-
    Pos1 \= Pos2,
    write('Monkey moves from '),
    write(Pos1),
    write(' to '),
    write(Pos2),
    nl.

push(state(Pos, onfloor, Pos, Has),
```

```

    state(middle, onfloor, middle, Has)) :-
Pos = middle,

write('Monkey pushes the box to the middle. '), nl.
climb(state(middle, onfloor, middle, Has),

    state(middle, onbox, middle, Has)) :-

write('Monkey climbs onto the box. '), nl.
grab(state(middle, onbox, middle, hasnot),

    state(middle, onbox, middle, has)) :-

write('Monkey grabs the bananas. '), nl.
solve :-

write('Monkey Banana Problem'), nl,

write('Monkey moves to the box. '), nl,

write('Monkey pushes the box below the bananas. '), nl,

write('Monkey climbs onto the box. '), nl,

write('Monkey grabs the bananas. '), nl.

```

Query : ?- solve.

Output:

```

56 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 25.pl']).
true.

57 ?- solve.
Monkey Banana Problem
Monkey moves to the box.
Monkey pushes the box below the bananas.
Monkey climbs onto the box.
Monkey grabs the bananas.
true.

```

Result : Thus, the Monkey Banana Problem was successfully implemented in Prolog.

Experiment 26. Prolog Program for Fruit and Its Color Using Backtracking

Aim: To write a Prolog program to find the color of different fruits using backtracking.

Algorithm

1. Start the program.
2. Define facts containing fruits and their colors.
3. Use the predicate fruit_color(Fruit, Color).
4. Query for a fruit or color.
5. Prolog uses backtracking to find all matching facts.
6. Display the results.
7. Stop.

Program :

```
fruit_color(apple, red).  
fruit_color(apple, green).  
fruit_color(banana, yellow).  
fruit_color(orange, orange).  
fruit_color(grapes, green).  
fruit_color(grapes, purple).  
fruit_color(mango, yellow).  
fruit_color(strawberry, red).
```

Query 1 – Find the color of apple

```
?- fruit_color(apple, Color).
```

```
?- fruit_color(Fruit, red).
```

Output :

```
58 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 26.pl']).  
true.
```

```
59 ?- fruit_color(apple, Color).  
Color = red ;  
Color = green.
```

```
60 ?- fruit_color(Fruit, red).  
Fruit = apple ;  
Fruit = strawberry.
```

```
61 ?- ▲
```

Result:

Thus, the fruit and color database using backtracking was successfully implemented in Prolog.

Experiment 27. Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm using heuristic values.

Algorithm

1. Start from the initial node.
2. Store the nodes along with their heuristic values.
3. Select the node having the lowest heuristic value.
4. Expand the selected node.
5. Repeat the process until the goal node is reached.
6. Display the path from the start node to the goal node.

Program:

edge(a,b).

edge(a,c).

edge(b,d).

edge(b,e).

edge(c,f).

edge(c,g).

edge(e,h).

edge(f,h).

heuristic(a,7).

heuristic(b,6).

heuristic(c,5).

heuristic(d,4).

heuristic(e,3).

heuristic(f,2).

heuristic(g,6).

heuristic(h,0).

best_first(Start,Goal,Path) :-

best_search(Start,Goal,[Start],RevPath),

reverse(RevPath,Path).

best_search(Goal,Goal,Visited,Visited).

best_search(Current,Goal,Visited,Path) :-

findall(H-Next,

(edge(Current,Next),

\+ member(Next,Visited),

heuristic(Next,H)),

List),

List \= [],

choose_best(List,Next),

best_search(Next,Goal,[Next|Visited],Path).

choose_best([_N],N).

choose_best([H1-N1,H2-N2|Rest],Best) :-

H1 <= H2,

choose_best([H1-N1|Rest],Best).

choose_best([H1-N1,H2-N2|Rest],Best) :-

H1 > H2,

choose_best([H2-N2|Rest],Best).

Query:

?- best_first(a, h, Path).

Output


```
62 ?- best_first(a, h, Path).
Path = [a, c, f, h] ;
false.
```

```
63 ?- best_first(a, h, Path).
Path = [a, c, f, h]
```

Possible actions:

;	(n,r,space,TAB):	redo		t:	trace&redo	
*	:	show choicepoint		.	(c,a,RET): stop	
w	:	write		p	:	print
+	:	max_depth*5		-	:	max_depth//5
b	:	break		h	(?):	help

Action? .

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

Experiment 28. Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to implement a simple medical diagnosis system based on symptoms.

Algorithm

1. Start the program.
2. Define symptoms for different diseases.
3. Define rules connecting symptoms with diseases.
4. Query the symptoms of a patient.
5. Use the rules to identify the possible disease.
6. Display the diagnosis.
7. Stop.

Program

```
symptom(ravi, fever).
```

```
symptom(ravi, cough).
```

```
symptom(ravi, body_pain).
```

```
symptom(priya, fever).
```

```
symptom(priya, sneezing).
```

```
symptom(priya, running_nose).
```

```
symptom(anu, stomach_pain).
```

```
symptom(anu, vomiting).
```

```
diagnosis(Person, flu) :-
```

```
    symptom(Person, fever),
```

```
    symptom(Person, cough),
```

```
    symptom(Person, body_pain).
```

```
diagnosis(Person, common_cold) :-
```

```

symptom(Person, fever),
symptom(Person, sneezing),
symptom(Person, running_nose).
diagnosis(Person, food_poisoning) :-
    symptom(Person, stomach_pain),
    symptom(Person, vomiting).

```

Query 1

```
?- diagnosis(ravi, Disease).
```

Output

```

64 ?- diagnosis(ravi, Disease).
Disease = flu .

65 ?- diagnosis(ravi, Disease).
Disease = flu
Possible actions:
; (n,r,space,TAB): redo          | t:          trace&redo
*:                show choicepoint | . (c,a,RET): stop
w:                write           | p:          print
+:                max_depth*5     | -:          max_depth//5
b:                break           | h (?):      help

Action?

```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

Experiment 29. Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and demonstrate the required queries.

Algorithm

1. Start with known facts.
2. Check the rules whose conditions match the known facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the required conclusion is obtained or no new fact can be derived.
6. Display the result.

Program

```
fact(sunny).
```

```
fact(weekend).
```

```
rule(go_out) :-
```

```
    fact(sunny),
```

```
    fact(weekend).
```

```
rule(play_cricket) :-
```

```
    fact(go_out).
```

```
derive(X) :-
```

```
    rule(X),
```

```
    assertz(fact(X)),
```

```
    write(X),
```

```
    write(' is derived. '), nl.
```

Query 1

?- rule(go_out).

Output

```
65 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 29.pl']).  
true.  
  
66 ?- rule(go_out).  
true.
```

Result

Thus, the Forward Chaining concept was successfully implemented in Prolog by deriving new facts from existing facts and rules.

Experiment 30. Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining and demonstrate the required queries.

Algorithm

1. Start with a goal.
2. Check whether the goal is a known fact.
3. If it is not a fact, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are true, the original goal is true.
7. Display the result.

Program

rain.

cloudy.

wet_ground :-

rain.

carry_umbrella :-

rain,

cloudy.

go_out :-

carry_umbrella.

Query 1

?- wet_ground.

?- carry_umbrella.

?- go_out.

Output

```
67 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 30.pl']).  
true.  
  
68 ?- wet_ground.  
true.  
  
69 ?- carry_umbrella.  
true.  
  
70 ?- go_out.  
true.  
  
71 ?- ▲
```

Result

Thus, the Backward Chaining mechanism was successfully demonstrated by starting from the goal and checking the required conditions.

